MERN Stack Course – Session 1

My First Express Server: The Blog API

Dr. Abdelweheb GUEDDES – École Polytechnique de Sousse – 2026–2027

Objective Build an HTTP-compliant backend server, test endpoints using a web browser and Postman, and master essential modern JavaScript idioms.

Prerequisites Core JavaScript literacy (variables, conditionals, functions, arrays) and basic command-line navigation.

Outline 1. Environment 2. Client, Server, and HTTP 3. Guided Lab: Blog API 4. Assignment 5. Summary

Submission Single PDF lab report (Section 4), **strictly due before 11:59 PM on the eve of next session**.

How to work with this guide. You will construct your backend **in progressive micro-steps**. Each step follows three clear phases: **Predict** (write down your expected behavior **before** executing code), **Your Turn** (write or adapt the implementation), **Checkpoint** (verify output before advancing). **JavaScript Concepts** are introduced exactly when required by the implementation. Making mistaken predictions is normal and encouraged: comparing expectations with actual behavior cements lasting understanding. Keep track of all predictions for your final report.

1 Environment Setup

Install the following tools using standard default configurations:

- **Node.js**, **LTS** release (`https://nodejs.org/`): executes JavaScript outside the browser to power our server runtime.
- **VS Code** (`https://code.visualstudio.com/`): modern code editor.
- **Postman** (`https://www.postman.com/downloads/`): API client for crafting and sending arbitrary HTTP requests.
- **Git** (`https://git-scm.com/`): distributed version control system.

Open a **new** terminal (PowerShell on Windows, or standard Terminal on Linux/macOS) and execute:

```
node -v
npm -v
```

Checkpoint. Two version identifiers should display (e.g., `v24.x.x` and `11.x.x`). **npm** is installed alongside Node.js to manage third-party libraries such as Express. If your shell returns "`node` is not recognized", restart your terminal. On Windows, if PowerShell restricts script execution, run: `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`

2 Client, Server, and HTTP

2.1 A Two-Step Conversation

On the web, all interactions form a structured conversation between a **client** and a **server**. The client (browser, Postman, or later a React application) emits an HTTP **request**. The server processes the request, performs the relevant operations, and returns an HTTP **response**. The formal communication contract governing this exchange is **HTTP** (*Hypertext Transfer Protocol*).

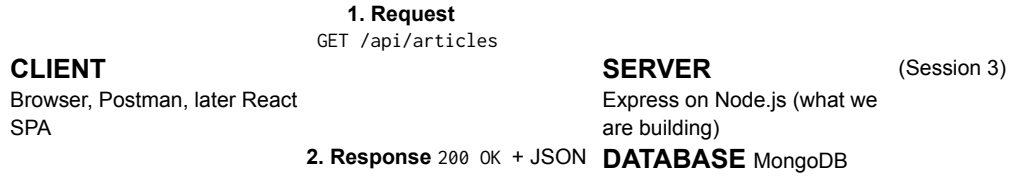


Figure 1: The client issues a request; the server answers with a response. In this session, data is kept in server memory; MongoDB persistence is introduced in Session 3.

> **The Restaurant Analogy.** You (the client) place an order with the waiter (the request). The waiter returns with the meal (the response). If the dish is unavailable, the waiter informs you politely rather than vanishing: this is precisely the role of **HTTP status codes**.

2.2 A Request = An HTTP Method + A Target URL

The HTTP **method** declares the intended action; the **URL** identifies the targeted entity. A **REST** (*Representational State Transfer*) API structures endpoints strictly around **resources** (e.g., articles) rather than procedural actions.

Method URL Client Intent Success Status

`GET /api/articles` "Fetch the collection of articles." `200 OK` `GET /api/articles/2` "Fetch article with identifier 2." `200 OK` `POST /api/articles` "Create a new article with the supplied payload." `201 Created` *Upcoming sessions:* `PUT` for complete replacement, `DELETE` for removal.

When processing fails or encounters issues, the server returns an explicit status code: `400 Bad Request` (client supplied invalid or incomplete data), `404 Not Found` (the requested resource does not exist), or `500 Internal Server Error` (unhandled server exception). Remember the code families: **2xx** denotes success, **4xx** denotes a client-side error, and **5xx** denotes a server-side error.

2.3 Anatomy of a URL

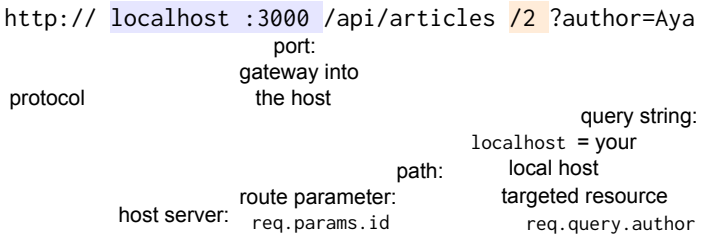


Figure 2: URL segments and their corresponding Express extraction properties.

Predict. In the URL `http://localhost:3000/api/articles/2`, does the segment `2` arrive inside the server as a **number** or as a **string**? Record your prediction: you will test it experimentally in Step 4.

3 Guided Lab: The Blog API

Step 1: Initialize the Project

In your terminal, execute:

```
mkdir my -blog -api
cd my -blog -api
npm init -y
npm install express
git init
code .
```

• `npm init -y` generates `package.json`, the project identity manifest.
• `npm install express` downloads **Express**, the web framework that abstracts low-level HTTP primitives. Dependencies are stored in `node_modules/`.
• `code .` opens the project directory in VS Code.

Inside VS Code, create a `.gitignore` file containing the line `node_modules/`: this directory contains thousands of installed dependencies generated on demand via `npm install`; it must never be checked into Git or submitted in archives.

Next, open `package.json` and configure the `"scripts"` block:

```
" scripts ": {
  "dev": " node --watch server .js"
}
```

From now on, running `npm run dev` starts the server, and Node's built-in `--watch` flag automatically reloads the process whenever a file is saved.

Step 2: A Minimal Server

➥ File `server.js` – run with `npm run dev`

```
const express = require ('express ') ; // 1. load the Express framework module

const app = express () ; // 2. instantiate the server application const PORT = 3000;

// 3. route definition : executed when a client requests GET /
app . get('/', (req , res) = > {
  res . json ({ message : "Hello , I am the blog API" }) ;
}) ;

// 4. bind and listen : await incoming HTTP requests on port 3000
app . listen (PORT , () = > {
  console . log(‘Server running at http :// localhost :${ PORT }‘) ;
}) ;
```

The foundational building block is the **route declaration** (3). Let us examine its structure: 3

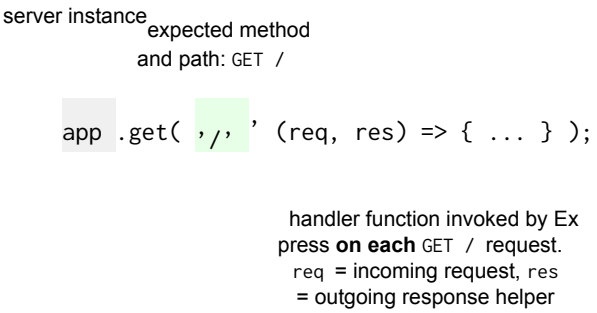


Figure 3: Anatomy of an Express route: "for this HTTP method and URL path, execute this handler function."

JavaScript Concept: Arrow Functions. `(req, res) => { ... }` is an **arrow function**: a concise modern syntax widely used for callback handlers. The two definitions below are functionally equivalent:

```
function greet ( name ) { return 'Hello ' + name ; } // traditional function const
greet2 = ( name ) = > { return 'Hello ' + name ; }; // arrow function
```

Here, the function is passed anonymously directly to `app.get`, which registers it and invokes it automatically whenever matching requests arrive.

JavaScript Concept: Template Literals. In `console.log(`...${PORT}`)`, the **backtick** de limiters (`) enable variable interpolation via `${...}`. This avoids clumsy string concatenation using the `+` operator.

Execute `npm run dev` in your terminal, then navigate to `http://localhost:3000` in your web browser.

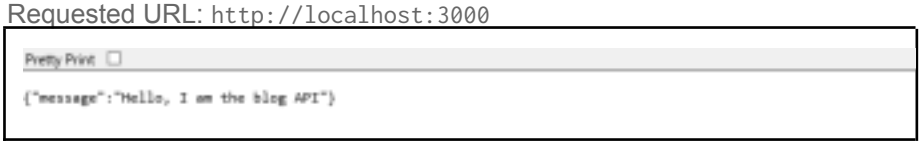


Figure 4: Browser rendering for `http://localhost:3000`. The browser acts as an HTTP client: it issued `GET /` and rendered the returned JSON payload.

Checkpoint. The terminal displays `Server running at http://localhost:3000` and the

browser displays the JSON payload. Modify the greeting text, save the file, and refresh your browser: the updated message appears instantly. Commit your progress: `git add .` followed by `git commit -m "Minimal server"`.

Step 3: The Article Collection

A blog manages articles. We store them initially in an in-memory JavaScript array. Insert this block **before** `app.listen(...)`:

```
// In - memory data store . State resets to defaults upon process restart . //
Persistent storage will be handled by MongoDB in Session 3.
const articles = [
  { id: 1 , title : 'Welcome to the blog ', author : 'Admin ' } ,
  { id: 2 , title : 'My first Express server ', author : 'Aya ' } ,
  { id: 3 , title : 'Testing an API with Postman ', author : 'Aya ' }
];

// GET / api/ articles -> fetch all articles
app . get('/api/ articles ', (req , res) = > {
```



```
  res . json ({ total : articles . length , articles : articles }) ;
}) ;
```

Requested URL: `http://localhost:3000/api/articles`

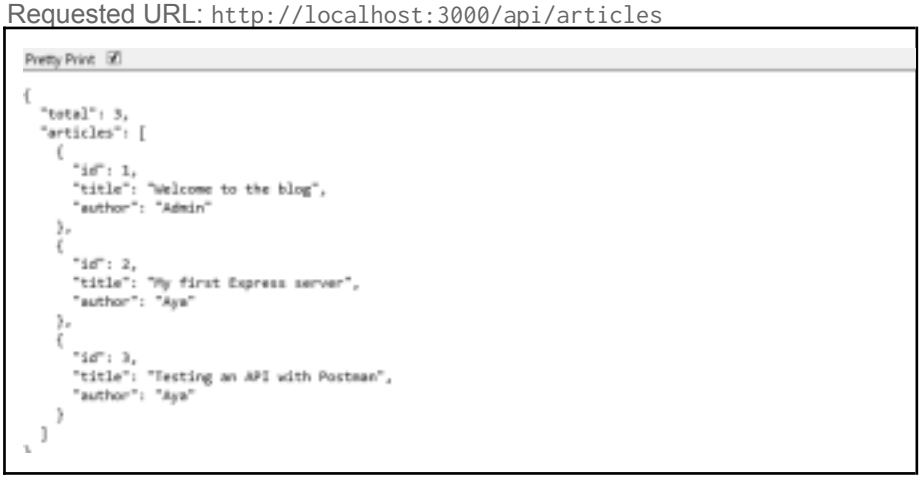


Figure 5: Response from `GET /api/articles` in the browser: a JSON object containing the total count and the articles array.

Checkpoint. Navigating to `http://localhost:3000/api/articles` displays `"total":3` along with the three articles. The in-memory JavaScript array has been serialized to JSON via `res.json`.

Step 4: Fetch a Single Article by ID

To fetch article #2, the client issues `GET /api/articles/2`. The identifier segment varies dynamically: we declare it using the `:id` route parameter, and Express makes it available under `req.params.id` (Figure 2).

```
// GET / api/ articles /2 -> fetch article whose id equals 2
app . get('/api/ articles /: id ', (req , res) = > {
  const id = Number ( req. params .id) ; // convert "2" ( string ) -> 2 ( number ) const
  article = articles . find (a = > a.id === id) ;

  if (! article ) {
    return res. status (404) . json ({ error : `Article ${id} not found ` }) ;
  }
  res . json ( article ) ;
}) ;
```

JavaScript Concept: `find`. `array.find(predicate)` iterates across elements and returns the **first** item satisfying the condition, or `undefined` if none matches. The predicate is a callback function returning a boolean.

```
const numbers = [5 , 12 , 8];
const large = numbers . find (n = > n > 10) ; // 12
const missing = numbers . find (n = > n > 100) ; // undefined
```

In our route handler, `a` represents an individual article, and the condition evaluates whether its `id` matches the requested parameter.

Your Turn. Experiment: Replace `Number(req.params.id)` with `req.params.id`, save the file, and open `/api/articles/2`. What happens? Compare with your initial prediction from Section 2: does `"2" === 2` evaluate to true? Restore `Number(...)` once tested.

Requested URL: `http://localhost:3000/api/articles/99`

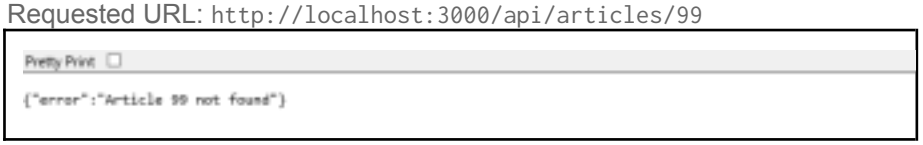


Figure 6: Response for a missing resource: the server handles the error gracefully, returning a structured JSON error object and HTTP status `404`.

Checkpoint. Visiting `/api/articles/2` displays Aya's article, whereas `/api/articles/99` dis plays "Article 99 not found". Why must `return` precede `res.status(404)`? An HTTP exchange permits strictly **one single response**: `return` terminates handler execution immediately to pre vent sending multiple responses. Commit your code: `"GET routes"`.

Step 5: Filter by Query Parameter

We now support filtering so that `GET /api/articles?author=Aya` returns only articles written by Aya. Everything following the `?` delimiter is parsed into `req.query` (Figure 2). **Replace** the Step 3 route with:

```
// GET / api/ articles -> all articles
// GET / api/ articles ? author = Aya -> articles filtered by author
app . get('/api/ articles ', (req , res) = > {
  const { author } = req. query ; // equivalent to: const author = req. query . author ;

  let result = articles ;
  if ( author ) { // if query parameter ? author = was provided result = articles .
    filter (a = > a. author === author ) ;
  }
  res . json ({ total : result . length , articles : result }) ;
}) ;
```

JavaScript Concept: Object Destructuring. `const { author } = req.query;` binds a local variable `author` to `req.query.author`. This modern idiom extracts properties cleanly:

```
const student = { name : 'Ali ', department : 'Computer Science ' };
const { name , department } = student ; // extracts both properties simultaneously console .
log(name , department ) ; // Ali Computer Science
```

Destructuring is standard across professional MERN codebases; we apply it again in the next step.

JavaScript Concept: `filter`. `array.filter(predicate)` returns a **new array** containing all elements satisfying the condition (unlike `find`, which stops at the first single element).

```
const numbers = [5 , 12 , 8];
const overSix = numbers . filter (n = > n > 6) ; // [12 , 8]
const empty = numbers . filter (n = > n > 100) ; // [] : empty array , not undefined
```

Predict. What is the resulting value of `total` for each of the following URLs? Note your predictions, then verify inside your browser:

- `http://localhost:3000/api/articles`
- `http://localhost:3000/api/articles?author=Aya`
- `http://localhost:3000/api/articles?author=Omar`

Step 6: Create an Article via POST

Until now, the client only **read** data. To **create** a new article, the client submits data inside the HTTP request **body** formatted as JSON. Express requires a dedicated parsing middleware: insert this line **immediately after** `const PORT = 3000;`:

```
app . use( express . json () ) ; // parses incoming JSON bodies and populates req. body
```

Then append the `POST` route before `app.listen(...)`:

```
let nextId = 4; // next auto - incremented id (ids 1 , 2 , and 3 are already assigned )

// POST / api/ articles -> creates an article from { " title ": "..." , " author ": "..." } app .
post ('/api/ articles ', (req , res) = > {
  const { title , author } = req. body ; // destructuring ( Step 5)

  if (! title || ! author ) { // validation : both fields are strictly required return
  res. status (400) . json ({ error : " Title and author are required " }) ; }

  const newArticle = { id: nextId , title : title , author : author };
  nextId = nextId + 1;
  articles . push ( newArticle ) ;

  res . status (201) . json ({ message : 'Article created ', article : newArticle }) ;
}) ;
```

Why is `express.json()` required? Across TCP/IP networks, HTTP payloads travel as raw serialized text streams. This middleware intercepts incoming requests with a `Content-Type: application/json` header, parses the raw string into a structured JavaScript object, and exposes it under `req.body` for all downstream route handlers.

Step 7: Test with Postman

Standard web browsers can only issue `GET` requests from the navigation address bar. Testing `POST` endpoints with custom JSON bodies requires Postman.



Figure 7: Postman interface overview: (1) select HTTP method; (2) enter URL; (3) open Body tab; (4) check raw and select JSON; (5) send request; (6) inspect HTTP status and response payload.

Predict. Before submitting each request, predict the resulting HTTP status code in the final column (refer to Section 2). Then compare against the actual Postman response.

Method	URL	Payload (Body, raw, JSON)	Predicted Status
GET	/api/articles	—	. . . . .

```
POST /api/articles { "title": "My first    ..............
article", "author": "Alex" }

GET /api/articles/4 — ..............POST /api/articles { "title": "Without author"
} ..............GET /api/articles/99 — ..............
```

Checkpoint. All five test cases return expected status codes, and `GET /api/articles` now reports `"total":4`. Restart the server process (`Ctrl + C` then `npm run dev`) and reload: the total returns to 3. This behavior is expected: data resides in volatile RAM, illustrating why production applications require a persistent database. Commit your code: `"Article creation"`.

Troubleshooting Guide

Symptom	Diagnostic & Resolution
`Cannot destructure property 'title' of 'req.body'` (status `500`)	Ensure `app.use(express.json())` precedes your routes.
`Cannot set headers after they are sent`	Missing `return` keyword before `res.status(...)`.
Body not recognized as JSON. In Postman: select **Body**, check **raw**, and choose **JSON**.	
`Cannot GET /api/article`	Endpoint mismatch or typo: verify the exact URL and HTTP verb declared in your route.
`EADDRINUSE` (port already occupied)	3000: terminate it using `Ctrl + C` in its terminal.
Server does not reload upon saving	The file was started with `node server.js` instead of `npm run dev` (which activates `--watch`).
An earlier server instance is still bound to port	

4 Assignment: Lab Report

Exercise 1: Extend the API (implement in `server.js`, verify with Postman)

1. ★ `GET /about` returns an object containing the application name, your student name, and the release version.
2. ★ `GET /api/users` returns an array of three users (`id`, `name`, `email`) declared in an in-memory `users` constant.
3. ★★ `GET /api/users/:id` returns the matching user object, or returns `404` if absent (model after Step 4).
4. ★★ `POST /contact` accepts `{ "email": "...", "message": "..." }`. If any field is missing, return `400`; otherwise return `200` with `{ "message": "Thank you, your message has been received" }`.
5. ★★★ *Bonus:* `GET /api/users?name=Aya` filters the user collection by name (model after Step 5).

Exercise 2: Essential JavaScript Idioms

➥ File `exercices_js.js` – run with `node exercices_js.js`

```
const products = [
  { name : 'Keyboard ', price : 45 } ,
  { name : 'Monitor ', price : 320 } ,
  { name : 'Mouse ', price : 25 }
];
```

1. Using **destructuring**, unpack `name` and `price` from the first product and print them to console.
2. Using `find`, locate the item named `'Mouse'` and log its price.
3. Using `filter`, return and log all products with a price strictly below 100.
4. Write an **arrow function** `withDiscount` taking a price and returning that price discounted by 10%. Log the result of `withDiscount(320)`.

Expected terminal output:

```
Keyboard 45
25
[ { name : 'Keyboard ', price : 45 } , { name : 'Mouse ', price : 25 } ]
288
```

Lab Report Deliverables

Submit **one single individual PDF report**, organized in the following order:

1. **Predictions:** Document your predictions (Section 2, Steps 5 and 7), the actual observed out comes, and a concise sentence explaining any divergence.
2. **Exercise 2:** Your complete source code alongside its terminal execution output. 3. **Exercise 1:** For every endpoint, include your implementation snippet and Postman screenshots (for `/api/users/:id` and `/contact`, include both a success case **and** an error case), with a one sentence caption explaining each capture.
4. **Complete Codebase:** Link to your public Git repository, or include the full `server.js` code in an appendix. Never include `node_modules`.

Submission Deadline: The lab report must be submitted before 11:59 PM on the eve of next session.

5 Summary & Key Takeaways

Five Core Principles

1. The web is a conversation: the **client** sends a request (HTTP verb + URL, optional JSON body), and the **server** answers with a response (status code + JSON payload).
2. An Express route maps a method and path to a handler: `app.get('/url', (req, res) => { ... })`.
3. Clients transmit parameters across three channels: `req.params` (route segments, `/:id`), `req.query` (query string, `?`), and `req.body` (JSON payload parsed by `express.json()`).
4. The server must send **strictly one response** per incoming request, accompanied by the proper HTTP status: `200`, `201`, `400`, or `404`.
5. URL parameters arrive exclusively as **text strings**: use `Number(...)` before numeric operations or strict equality comparisons.

Goal Syntax Pattern

Register an endpoint route `app.get('/api/articles', (req, res) => { ... });` Return a JSON payload `res.json({ total: 3 });`

Return an explicit status code `return res.status(404).json({ error: '...' });`

Read route parameter `/api/articles/7`

Read query parameter `?author=Aya`

Read incoming `POST` JSON payload `app.use(express.json());` then `const { title } = req.body;`

```
const id = Number(req.params.id);

const { author } = req.query;
```

Locate a single matching element `array.find(a => a.id === id)` Filter multiple matching elements `array.filter(a => a.author === author)` Start / terminate local server `npm run dev` / `Ctrl + C`

Self-Assessment Checklist (check each item; review the indicated section if needed) □

Explain requests, responses, and standard HTTP status code families Section 2

□ Initialize a Node.js project and run an Express server with automatic reload Steps 1–2

□ Write a `GET` endpoint returning an array serialized to JSON Step 3 □ Extract `req.params` and `req.query`, and apply `find` and `filter` Steps 4–5 □ Implement a validated `POST` endpoint and verify it via Postman Steps 6–7 □ Apply arrow functions and object destructuring idioms JavaScript Concepts

Session Concept Reference

Concept	Purpose	Example
Arrow function	compact function syntax	`const double = x => x * 2;`
Template literal	string interpolation	`` `Port ${PORT}` ``
Destructuring	extract object properties cleanly	`const { title } = req.body;`
`find`	locate FIRST matching item (or `undefined`)	`articles.find(a => a.id === id)`
`filter`	extract all matching items	`articles.filter(a => a.author === author)`
`map` ★	transform each array element	`[1,2,3].map(n => n * 2)` yields `[2,4,6]`
`app.get`	declare a read endpoint handler	`app.get('/api/articles', ...)`
`app.post`	declare a resource creation handler	`app.post('/api/articles', ...)` must precede route definitions
`app.use(express.json())`	parse incoming JSON payloads into `req.body`	
`res.json(...)`	serialize and return JSON response	`res.json(article)`
`res.status(...)`	set HTTP response status code	`res.status(404).json(...)`
`Number(...)`	cast string representation to number	`Number(req.params.id)`

★ *Bonus:* `map` is the counterpart of `filter`: instead of filtering items, it transforms every element in an array into a new value. Test it inside `exercices_js.js`.

Looking Ahead. Our current server responds instantaneously because data resides directly in memory. When data is queried from a remote database, the server will need to **await** responses asynchronously without blocking other concurrent user requests: this is the essence of **asyn chronous JavaScript** (callbacks, Promises, and `async/await`), which we will master in Session 2 before connecting MongoDB in Session 3.